

The Redemption: Architecture & Operations Design Document

Zero-downtime, 10x-spike-tolerant AWS EKS platform for global hotel point deductions

Prepared for Accor - Hiring Panel

Prepared by Hardik Bhandari (Candidate, Cloud Engineer)

Scope AWS EKS compute, scalability, security, reliability, and Day-2 operations design

Companion artifacts GitHub repository (Terraform + Helm chart), architecture diagram

Contents

1. Executive Summary
2. Compute & Architecture
3. Scalability Strategy
4. Security & Networking
5. Reliability & Observability
6. Day-2 Operations & Toil Reduction
7. Team Delegation Plan
8. Key Trade-offs Summary

1. Executive Summary

The Redemption deducts loyalty points across Accor's global hotel portfolio. Traffic is steady most of the time and spikes roughly 10x during flash sales, and any service failure converts directly into lost or miscounted revenue. The design in this document optimizes for one outcome above all others: **the service keeps deducting points correctly, even while infrastructure underneath it is actively failing or scaling.**

The architecture rests on four decisions that recur throughout every section below:

1. **Three availability zones, never two.** Losing one AZ during a flash sale must leave the service at 67% capacity, not 50%.
2. **Two separate compute lanes.** A small, fixed, on-demand "baseline" lane that is never touched by autoscaling, and a disposable, spot-heavy "burst" lane that Karpenter and KEDA scale aggressively for flash sales. The guaranteed floor and the cheap elastic capacity are never the same pool of nodes.
3. **Defense in depth as five concrete, independently-enforced layers** - edge WAF, network security groups, service-mesh mTLS/identity, workload-level Pod Security Standards, and data-at-rest encryption - rather than a single perimeter that, if breached, exposes everything behind it.
4. **SLO-driven operations**, not infrastructure-metric-driven operations. Alerts page on error-budget burn rate against a defined 99.95% availability SLO, not on raw CPU/memory thresholds that correlate poorly with customer-visible pain.

The full implementation is provided as working Terraform modules and a Helm chart in the accompanying GitHub repository, validated structurally against every cross-reference (resource names, module variables, label selectors, Helm template references) prior to submission. Section 8 of this document summarizes every major trade-off considered, including the alternative that was rejected and why.

2. Compute & Architecture

2.1 Network foundation

The VPC spans three AZs with a three-tier subnet model: **public** (ALB + one NAT Gateway per AZ), **private-app** (EKS worker nodes and pods), and **private-data** (RDS, ElastiCache - no internet route at all). Each AZ's app subnet egresses through its own AZ's NAT Gateway, so a single NAT Gateway failure is contained to one AZ instead of taking down egress cluster-wide.

Why one NAT Gateway per AZ instead of one shared NAT Gateway: a single NAT Gateway is a textbook hidden single point of failure - every outbound call (image pulls, the payment gateway callback, AWS API calls) routes through it. For a revenue-critical service the ~3x cost of per-AZ NAT Gateways is the correct trade against an unscoped, cluster-wide egress outage.

2.2 Two-lane compute model

The EKS cluster runs two independently-managed sets of worker nodes, expressed as two separate Kubernetes Deployments behind one Service:

Lane	Managed by	Capacity type	Replica range	Role
Baseline the-redemption	Fixed managed node group	On-demand only	Fixed at 3 (1 per AZ)	Guaranteed floor. Never touched by KEDA. Still standing if Karpenter or spot capacity has a bad day.
Burst the-redemption-burst	Karpenter	Spot + on-demand mix	0–40, KEDA-driven	Absorbs the 10x flash-sale spike. Disposable by design, tolerates a dedicated taint so baseline pods can never accidentally land on spot capacity.

This separation is the architectural backbone of the entire reliability story: it means "guaranteed availability" and "cost-efficient elasticity" are never in tension on the same set of nodes, because they are not the same set of nodes.

2.3 Pod-level resilience

Within each lane, three Kubernetes primitives work together to survive node and AZ failure:

- **Hard pod anti-affinity** (baseline lane) - two replicas never share a node, so losing one node costs exactly one replica, never two.
- **Required topology spread across AZs** (baseline lane) - `whenUnsatisfiable: DoNotSchedule` guarantees one replica per AZ rather than merely preferring it.
- **PodDisruptionBudget** (`minAvailable: 2 of 3`) - protects against voluntary disruption (a cluster upgrade, a Karpenter consolidation) on top of the involuntary-failure protection above. Without it, a well-intentioned node rotation could evict two of three baseline replicas at once.

2.4 Why EKS over ECS or a self-managed cluster

Chosen: Amazon EKS

The assessment scenario specifies EKS directly. Beyond that constraint, EKS gives access to the broader Kubernetes ecosystem this design leans on heavily - Karpenter, KEDA, Istio, and the kube-prometheus-stack all assume Kubernetes APIs and would need to be reimplemented or substituted on ECS.

Considered: ECS Fargate

Simpler operationally (no node management at all), but loses fine-grained scheduling control - no Karpenter-style bin-packing against pending pod shapes, no Istio sidecar mesh, and KEDA's Kubernetes-native scaling primitives. Would have been the right call for a simpler, less spike-prone service.

3. Scalability Strategy

The 10x flash-sale requirement is solved at two layers that must work together: KEDA decides *how many pods* are needed, Karpenter decides *how much node capacity* exists for those pods to run on. Neither alone is sufficient - KEDA without fast node provisioning just creates a backlog of Pending pods, Karpenter without an application-aware scaling signal has nothing pending to react to.

3.1 Application-level: KEDA over plain HPA

Capability	Plain HPA (CPU/memory)	KEDA (this design)
Signal type	Lagging - CPU rises after requests are already queuing	Leading - scales on live request-rate from Istio's sidecar metrics, before CPU saturates
Scale to zero	Not supported	Supported - the burst lane sits at zero replicas (and zero cost) outside of flash sales
External metrics	Custom metrics adapter required, non-trivial to operate	Native triggers for Prometheus queries and SQS queue depth out of the box

The ScaledObject's scale-up policy is deliberately aggressive (can double pod count every 30 seconds, uncapped delta) while the scale-down policy is deliberately conservative (max 10% of pods removed per minute, 120-second stabilization window). Real flash sales often arrive in waves - an initial rush, a lull, a second wave when a discount code goes viral on social media - and scaling down too eagerly after the first wave risks thrashing into the second.

3.2 Infrastructure-level: Karpenter over Cluster Autoscaler

Cluster Autoscaler scales pre-defined Auto Scaling Groups of a fixed instance type, reacting to scheduling failures relatively slowly. Karpenter instead reasons directly from the resource shape of *pending* pods to the cheapest currently-available EC2 instance type across several families, and typically provisions a new, right-sized node in under 60 seconds. For a sudden 10x spike, that difference in provisioning latency is the difference between "customers wait a few seconds longer during the very first burst" and "customers see errors."

Cost containment: the Karpenter NodePool sets a hard ceiling (`limits: cpu: 200`) so a runaway scaling loop - a bug, a retry storm, an actual DDoS dressed up as a "flash sale" - has a worst-case bill, not an unbounded one. `consolidationPolicy: WhenEmptyOrUnderutilized` with a 60-second window means we stop paying for spike-sized capacity within a minute of the spike ending.

3.3 Database scaling

RDS PostgreSQL runs Multi-AZ with one read replica. The application is expected to route point-balance reads (the read-heavy part of flash-sale traffic) to the replica and route point-deduction writes (the

business-critical, must-be-consistent part) to the primary. RDS storage autoscaling (100 GiB → 500 GiB ceiling) removes "did we run out of disk during a sale" as an on-call concern. ElastiCache Redis, Multi-AZ with automatic failover, absorbs idempotency-key lookups and hot point-balance reads off the database entirely - this is frequently the actual bottleneck in spike scenarios, not application CPU.

4. Security & Networking

"Defense in depth" is implemented as five layers that are each independently enforced - a failure or bypass at one layer does not collapse the guarantee, because no single layer is solely load-bearing for the whole posture.

Layer	Mechanism	What it stops
1. Edge	AWS WAF (managed rule groups + a per-IP rate-based rule) in front of the ALB	Common web exploits, known bad IPs, and - specifically tuned for this scenario - scraper/bot traffic that mimics a flash-sale spike but concentrates from few IPs, while genuine multi-customer flash-sale traffic passes through untouched
2. Network	Security groups, default-deny with explicit allow rules per port/source-SG. The data-layer security group has zero egress rules - RDS and ElastiCache can never initiate an outbound connection	Lateral movement at the network layer, a compromised pod cannot reach anything it wasn't explicitly granted a path to
3. Mesh / identity	Istio service mesh: mesh-wide STRICT mTLS plus AuthorizationPolicy scoped to specific workload identities (e.g. only the ingress gateway's own ServiceAccount may call The Redemption)	A different compromised service inside the same cluster calling this one - identity-based, not just IP-based, so it holds even if two services share a subnet
4. Workload	Pod Security Standards "restricted" enforced at the namespace (admission-controlled, not just a checklist): non-root, no privilege escalation, read-only root filesystem, all Linux capabilities dropped	Container breakout and privilege escalation even if application code is compromised
5. Data	KMS customer-managed keys for EKS secrets, RDS, and EBS volumes, RDS master credentials generated and stored by AWS Secrets Manager directly (never visible in Terraform state, code, or a human's clipboard), application secrets synced via External Secrets Operator	Data exposure even given disk or snapshot access, eliminates plaintext credentials from the entire pipeline

4.1 Least privilege via IRSA

Every component that needs AWS API access - Karpenter, the AWS Load Balancer Controller, External Secrets, the application itself - gets its own narrowly-scoped IAM role bound to its own Kubernetes ServiceAccount (IAM Roles for Service Accounts). This is the deliberate alternative to a single fat node IAM role inherited by every pod on a node: a compromised application pod cannot use Karpenter's EC2 permissions, because it was never granted them in the first place.

4.2 IMDSv2 enforcement

Karpenter-provisioned nodes set `httpTokens: required` and `httpPutResponseHopLimit: 1`, closing the SSRF-to-instance-credential-theft path that IMDSv1 leaves open - a detail that's easy to miss on dynamically-provisioned nodes specifically because they're created outside the original launch-template review.

5. Reliability & Observability

5.1 Service Level Objective

The Redemption's SLO: **99.95% of point-deduction requests succeed within 500ms**, measured over a rolling 30-day window. That budget is roughly 21.6 minutes of full-downtime-equivalent per month - small enough to take seriously, large enough to allow a controlled deploy or a brief AZ failover without breaching it.

5.2 Alerting: burn rate, not raw thresholds

Alerts use Google SRE's multi-window, multi-burn-rate method rather than a flat "error rate > X% for Y minutes" rule. A single fixed window is either too noisy on short blips or too slow to catch a real incident. Pairing a short window (5 minutes, catches fast severe burns) with a long window (1 hour, confirms the burn is sustained, not a blip) before paging gives a much better signal-to-noise ratio for an on-call rotation of three engineers who cannot afford alert fatigue.

- **Fast burn** (14.4x rate, 5m+1h sustained) → pages immediately. At this rate the entire 30-day budget would be gone in about 2 days.
- **Slow burn** (3x rate, 6h sustained) → opens a ticket, not a page. Real problem, not a 2am emergency.

5.3 Recovering from specific failure modes

Failure	Detection	Recovery	Customer-visible impact
Single pod crash	Liveness probe failure	Kubernetes restarts the container automatically	None - readiness probe already pulled it from rotation before it failed liveness
Single node failure	Node NotReady, kubelet heartbeat lost	Anti-affinity means only one replica was on that node, remaining replicas absorb traffic, Karpenter or the ASG replaces the node	Negligible - circuit breaker (below) ejects any pod returning errors during the brief gap
AZ outage	Multiple node NotReady in one zone, RDS Multi-AZ failover event	Topology spread means the other 2 AZs still hold baseline replicas, RDS fails over to its standby automatically, ElastiCache fails over via Multi-AZ	Brief (sub-minute) latency blip during RDS failover, no full outage
Bad deployment	Readiness probe failures spike post-rollout, or SLO burn-rate alert fires	maxUnavailable: 0 rolling update means a bad new pod never reduces available capacity, ArgoCD rollback (Section 6) reverts to last-known-good	None for the fleet, the bad revision simply never receives traffic if readiness fails
One pod degraded under load	Istio outlier detection: 5 consecutive 5xx responses	DestinationRule ejects the pod from the load-balancing pool for 30s without killing it	None - traffic silently routes around the struggling pod

5.4 The three-probe model

Each pod defines a **startup**, **readiness**, and **liveness** probe with deliberately different sensitivities. The liveness probe is intentionally lenient (6 consecutive failures over a minute before restart) - an aggressive liveness probe is a well-known self-inflicted-outage pattern, where a pod that's merely slow under genuine load gets killed by Kubernetes at exactly the moment it's needed most. The readiness probe, by contrast, fails fast (2 failures, 2-second timeout) because pulling a struggling pod out of rotation without restarting it is cheap and reversible.

6. Day-2 Operations & Toil Reduction

6.1 GitOps via ArgoCD

Application and addon changes flow through Git, not through engineers running `kubectl apply` or `helm upgrade` from a laptop. ArgoCD continuously reconciles the cluster against the `helm/the-redemption` chart and its `values.yaml` in this repository - a deliberate choice over a plain manifest tree, since this is one service with one base configuration: a future second environment is a small `values-staging.yaml` layered on top via ArgoCD's `valueFiles`, not a parallel near-duplicate manifest tree that silently drifts from the original. The practical toil reduction: no engineer needs standing `kubectl` write access to the production cluster for routine changes, no one needs to remember which laptop has the right `kubeconfig`, and every change has an automatic audit trail (the git log) without anyone having to maintain one by hand.

Why a Helm chart, specifically. Plain manifests with Kustomize overlays would also have worked for a single service with one environment - this is not a case where the alternative was wrong. The chart was chosen because the baseline and burst Deployments share almost everything (probes, security context, resource shape) and differ only in a handful of fields (node selector, tolerations, replica ownership). Helm's `_helpers.tpl` expresses that shared logic once, in one named template, so a probe-path change is made in a single place instead of kept in sync by hand across two near-duplicate YAML files. The trade-off accepted in exchange is template readability: a reviewer has to mentally resolve `{{ include }}` calls rather than reading a flat YAML file top to bottom.

6.2 What gets automated vs. what stays a human decision

Automated	Stays manual / human-gated
Pod scaling (KEDA), node scaling (Karpenter)	Raising the <code>maxReplicaCount</code> ceiling itself - alerted via <code>RedemptionHPAMaxedOut</code> , decided by a human, not silently raised
Certificate issuance/rotation (cert-manager + Istio)	DNS cutover for a new hostname
Secret rotation pickup (External Secrets, 1h refresh)	The decision to rotate a credential in Secrets Manager itself
Routine deploys (ArgoCD auto-sync from <code>main</code>)	Approving the PR that merges to <code>main</code> - the gate is code review, not a deploy button
Spot interruption handling (Karpenter + SQS notice, ~2 min warning)	Choosing instance type families available to the burst NodePool

6.3 Reducing toil further

- **Runbooks linked directly from alerts** (see the runbook annotation on every PrometheusRule) - an on-call engineer at 3am should never need to reconstruct context from scratch.
- **No `:latest` image tags, ever.** Every deployment references an immutable Git SHA, so "what's actually running in production" is never a question that requires SSHing into a node to answer.

- **Load testing before every flash sale**, not just once at launch. The KEDA scale-up threshold (50 in-flight requests per pod) and Karpenter's instance-type list should be revisited quarterly as real traffic patterns and the catalog of hotel promotions evolve - this is explicitly a recurring task in Section 7's delegation plan, not a one-time setup step.

7. Team Delegation Plan

Implementation work is split by **blast radius and reversibility**, not just by topic. The senior engineer owns anything that's hard to reverse or that gates other people's work, juniors own well-scoped, reversible, independently-testable pieces with clear interfaces - deliberately structured as a learning path, not just task assignment.

Engineer	Owns	Why this split
Senior	VPC + security module design and review, EKS cluster + IRSA role boundaries, Istio mTLS/AuthorizationPolicy rules, final review and merge approval on every junior PR, the two-pass Terraform apply for initial bootstrap, on-call escalation point for fast-burn pages	These decisions are expensive to reverse once live (re-architecting a VPC CIDR plan after RDS is provisioned is a real outage risk) or define the security boundary everything else operates inside. Mistakes here have the largest blast radius.
Junior 1	Kubernetes application manifests (Deployment, Service, PDB, ConfigMap/ExternalSecret), KEDA ScaledObject tuning against real load-test results, PrometheusRule alert thresholds, Grafana dashboards	High learning value (directly touches the app's actual runtime behavior), fully reversible via <code>kubectl rollout undo</code> or a Git revert, and testable in isolation against a staging namespace before touching production.
Junior 2	Karpenter NodePool/EC2NodeClass tuning (instance type lists, spot/on-demand ratios), WAF rule tuning and rate-limit threshold calibration, CI/CD pipeline (image build, SHA tagging, ArgoCD sync triggers), the structural validator scripts and their integration into CI	Infrastructure-adjacent but bounded blast radius - a bad Karpenter instance-type list degrades cost efficiency, it doesn't take down the baseline floor. Strong exposure to both the cost/performance trade-off space and to building the automation the team will lean on daily.

All three engineers review each other's PRs, the senior engineer holds final merge approval specifically on anything touching `terraform/modules/vpc`, `terraform/modules/security`, or the Istio mTLS configuration, since those are the pieces where a quiet misconfiguration is most likely to surface as a production incident rather than a CI failure.

8. Key Trade-offs Summary

Decision	Chosen	Rejected alternative	Why
Node autoscaler	Karpenter	Cluster Autoscaler	Reacts to pending-pod shape directly, provisions in <60s vs. CA's slower ASG-metric-driven scaling
App autoscaler	KEDA	Plain HPA	Leading signal (request rate) instead of lagging (CPU), native scale-to-zero and external triggers (SQS)
Service mesh	Istio	Plain NetworkPolicy only	Requirement is encrypted east-west traffic + identity-based authorization, not just L3/L4 IP segmentation
NAT topology	One NAT Gateway per AZ	One shared NAT Gateway	Removes a cluster-wide single point of failure for all egress traffic, at ~3x cost - accepted for a revenue-critical service
Compute model	Two separate Deployments (baseline + burst)	One Deployment scaled wide	Keeps the guaranteed floor and disposable spot capacity from ever sharing a scaling unit or a node pool
Manifest packaging	Helm chart	Plain manifests / Kustomize overlays	Baseline and burst Deployments share probes, security context, and resource shape, a named helper template keeps that logic in one place instead of two near-duplicate YAML files kept in sync by hand
Alerting	SLO burn-rate (multi-window)	Flat infrastructure thresholds	Far better signal-to-noise for a 3-person on-call rotation, pages correlate with actual customer pain
Compute platform	EKS	ECS Fargate	Assessment-specified, also needed for the Karpenter/KEDA/Istio ecosystem this design relies on